Exosuit Design Meeting Notes

Technical Analysis of Key Discussion Points

`cable/cable_with_exo_springs*.py`

Isaac-sim robotics notes

April 2026

1 Context

These notes document the design discussions around the exosuit co-contraction mechanism for the cup manipulator. Each point raised in the meeting is stated, then assessed against the mathematical models derived in the companion document (`exosuit_stiffness_and_co_contraction.tex`).

Two cable routing methods are under consideration:

- **Method A** — offset elbow pulleys ($r_{\text{elb}} = 32$ mm, $d_{\text{off}} = 103$ mm);
- **Method B** — centred elbow pulley ($r_{\text{cp}} = 30$ mm from mesh, on joint axis).

2 Point 1: Two Separate Pulleys vs. One Pulley with Two Grooves

Meeting claim: "Use two separate elbow pulleys, one for each exo cable."

∼ **Partially correct / nuanced.** This describes **Method A**, which does use two separate elbow pulleys (right and left, offset from the joint axis). However, the current Method B implementation in `cable_with_exo_springs_elbow_follow.py` uses a **single shared pulley** with **two grooves** separated by 3 mm (upper at $Z = 286.55$ mm, lower at $Z = 283.55$ mm).

Both approaches are valid but have different trade-offs:

- **Two separate pulleys** (Method A): simpler tangent geometry (no groove-level Z tracking), but variable moment arm and $d_{\text{off}} \neq 0$.
- **Single pulley, two grooves** (Method B): constant moment arm (r_{cp}), exact cable kinematics, but requires careful Z-plane management and encoder tracking.

The code currently supports both. The choice depends on whether the prototype uses the centred design (Method B) or offset mounting (Method A).

3 Point 2: Stepper Motor Latency for Zero-Resistance Feel

Meeting claim: "A stepper motor can track the elbow encoder fast enough to maintain zero cable tension during free rotation."

∼ **Partially correct / nuanced.** The encoder-tracking law for Method B requires the motor to feed cable at:

$$\dot{l}_m = r_{\text{cp}} \cdot \dot{q}_2 \tag{1}$$

For a fast elbow motion $\dot{q}_2 = 5$ rad/s and $r_{\text{cp}} = 47.75$ mm:

$$\dot{l}_m = 0.04775 \times 5 = 0.239 \text{ m/s} = 239 \text{ mm/s}$$

Latency analysis: A typical NEMA-17 stepper at 3000 steps/s with 1.8/step and 20 mm pulley

gives $v_{\max} \approx 300$ mm/s — marginally sufficient. However:

1. **Step-direction latency**: ~ 1 ms (acceptable).
2. **Acceleration bandwidth**: Limited by rotor inertia and holding torque. During sudden direction changes ($\ddot{q}_2 > 50$ rad/s^2), the stepper may lose steps.
3. **Microstepping resonance**: At mid-range speeds (200–800 steps/s) steppers exhibit torque dips.

The spring extension due to tracking latency Δt:

$$\delta_{\text{lag}} = r_{\text{cp}} \cdot \dot{q}_2 \cdot \Delta t \tag{2}$$

For $\Delta t = 5$ ms and $\dot{q}_2 = 5$ rad/s: $\delta_{\text{lag}} = 0.04775 \times 5 \times 0.005 = 1.19$ mm. Parasitic force: $F = k_{\text{exo}} \times 0.00119 = 0.24$ N (with $k_{\text{exo}} = 200$ N/m).

Verdict: At slow motions ($\dot{q}_2 < 2$ rad/s), stepper tracking is adequate. At fast motions with sudden reversals, the parasitic force may be perceptible. A **servo motor with encoder feedback** (e.g., closed-loop stepper or small BLDC) would provide tighter tracking.

4 Point 3: CubeMars Impedance Control Eliminating Compliance

Meeting claim: "Using a CubeMars motor in impedance/torque mode could eliminate the need for physical springs entirely — the motor itself provides the compliance."

$\sim$ **Partially correct / nuanced.** This is conceptually attractive but has important limitations:

What CubeMars MIT torque mode provides:

- Direct torque control at ~ 1 kHz loop rate
- Programmable virtual stiffness: $\tau = K_p(\theta_{\text{des}} - \theta) + K_d(\dot{\theta}_{\text{des}} - \dot{\theta})$
- AK60-6: $\tau_{\text{peak}} = 9$ N m, $J_m = 3.32 \times 10^{-5}$ kg m^2, $N = 6$

Why physical springs are still valuable:

1. **Impact bandwidth:** An impact at the elbow has a characteristic time $\sim$ 1–5 ms. The CubeMars control loop runs at ~ 1 ms. A physical spring responds at the *speed of sound in the material* ($\ll 0.1$ ms) — orders of magnitude faster.
2. **Energy storage:** Springs absorb impact energy passively. A motor must dissipate energy electrically (back-EMF, resistive losses).
3. **Fail-safe:** If motor power is lost, a pre-tensioned spring still provides stiffness. A virtual spring provides zero stiffness when powered off.
4. **Motor resonance:** The reflected rotor inertia $J_{\text{eff}} = N^2 \cdot J_m = 36 \times 3.32 \times 10^{-5} = 1.2 \times 10^{-3}$ kg m^2 creates a resonance with the virtual spring:

$$\omega_n = \sqrt{\frac{K_p}{J_{\text{eff}}}} \tag{3}$$

   For $K_p = 10$ N m/rad: $\omega_n = \sqrt{10/0.0012} \approx 91$ rad/s (14.5 Hz). Above this frequency, the motor cannot track.

Verdict: CubeMars impedance mode is excellent for *low-frequency* stiffness modulation (trajectory following, gravity compensation) but **cannot replace physical springs for impact protection**. The recommended architecture is **hybrid**: physical springs for high-bandwidth passive compliance + CubeMars for active stiffness tuning. This is exactly the SEA (Series Elastic Actuator) design already implemented in `actuators/sea.py`.

5 Point 4: The "$2k$" Co-Contraction Stiffness

Meeting claim: "When both exo cables co-contract, the effective joint stiffness is $2k$ times some constant."

✓ **Correct.** This is exactly the co-contraction result derived in the companion document. For symmetric pretension on antagonistic cables, the effective stiffness at the joint is:

$$\boxed{k_{\text{eff}} = 2\, k_{\text{exo}}\, r_{\text{eff}}^2} \tag{4}$$

where the factor of 2 arises because *both* cables contribute to restoring torque (one cable gets longer, the other shorter, and both resist the deflection). The effective moment arm r_{eff} depends on the method:

- **Method A:** $r_{\text{eff}} = l_c'(q_2) \approx r_{\text{elb}} = 32$ mm $\Rightarrow$ $k_{\text{eff}}^A \approx 2 \times 200 \times 0.032^2 = 0.41$ N m/rad
- **Method B:** $r_{\text{eff}} = r_{\text{cp}} = 47.75$ mm $\Rightarrow$ $k_{\text{eff}}^B = 2 \times 200 \times 0.04775^2 = 0.91$ N m/rad

Derivation sketch: A deflection Δq from equilibrium changes each cable's spring extension by $\pm r_{\text{eff}} \cdot \Delta q$. Each spring contributes force $k_{\text{exo}} \cdot r_{\text{eff}} \cdot \Delta q$, and each force acts through moment arm r_{eff}, giving torque $k_{\text{exo}} \cdot r_{\text{eff}}^2 \cdot \Delta q$ per cable. Two cables $\Rightarrow$ factor of 2.

6 Point 5: Springs on the Upper Arm Side

Meeting claim: "Place the springs on the upper arm (link 1) side of the elbow pulley."

× **Needs correction.** In the current architecture, the springs are placed **between the elbow pulley and the link-2 anchor** — i.e., on the **forearm** (link 2) side, not the upper arm (link 1) side.

Why the forearm side is correct:

1. The cable segment from motor → L1 pulley → elbow pulley must be *inextensible* — the motor must control cable displacement precisely. A spring on this side would absorb motor motion and destroy position authority.
2. The spring on the link-2 side acts as a *force sensor*: spring extension δ directly indicates cable tension $F = k_{\text{exo}} \cdot \delta$.
3. From the TikZ architecture diagram (`exosuit_stiffness_and_co_contraction.tex`, **??**): the cable path is Motor → cable → L1 pulley → cable → Elbow pulley → **spring** → cable → End anchor (link 2).
4. The spring provides compliance at the *impact site* (link 2 interacts with objects), which is the correct place for impact absorption. A spring on the motor side would delay force transmission rather than absorb impacts locally.

In the code (`cable_with_exo_springs_elbow_follow.py`), the spring visual is drawn as a fraction (`spring_fraction` = 0.12) of the cable segment between the elbow pulley and the end anchor — i.e., on the link-2 side.

7 Point 6: Transmission Ratio via Pulley Diameters

Meeting claim: "Changing the motor pulley diameter relative to the elbow pulley changes the transmission ratio and thus the stiffness."

✓ **Correct.** The transmission ratio between motor and joint is determined by the pulley radii.

Define:

$$r_m = \text{motor pulley radius}, \tag{5}$$
$$r_{\text{elb}} = \text{elbow pulley radius (or } r_{\text{cp}} \text{ for Method B)}. \tag{6}$$

The kinematic relation is:

$$l_m = r_m \cdot \theta_m, \qquad l_c = r_{\text{elb}} \cdot q_2 \qquad \Rightarrow \qquad \frac{\theta_m}{q_2} = \frac{r_{\text{elb}}}{r_m} \tag{7}$$

If the motor and elbow pulleys are the same radius ($r_m = r_{\text{elb}} = r_{\text{cp}}$), the ratio is 1:1.

Effect on stiffness: The spring extension becomes:

$$\delta = r_m \cdot \theta_m - r_{\text{elb}} \cdot q_2 \tag{8}$$

The effective joint stiffness with co-contraction:

$$k_{\text{eff}} = 2\, k_{\text{exo}}\, r_{\text{elb}}^2 \tag{9}$$

Note: the elbow-side radius determines joint stiffness (it is the output lever arm). Changing r_m affects the *motor-side resolution and torque requirement*:

- **Smaller** r_m: motor sees higher mechanical advantage, needs less torque but more rotation per unit cable displacement.
- **Larger** r_m: motor sees lower mechanical advantage, needs more torque but less rotation.

Currently, Method B uses $r_m = r_{\text{cp}} = 47.75$ mm (1:1 ratio), which simplifies the encoder tracking law to $\theta_m = q_2 + \Delta\theta$.

8 Point 7: Encoder Tracking Necessity for Method B

Meeting claim: "Method B (centred pulley) requires continuous encoder tracking to prevent parasitic forces during free rotation."

✓ **Correct.** This is the fundamental requirement of Method B. Because the elbow pulley is *on* the joint axis, any rotation Δq_2 wraps/unwraps cable by exactly $r_{\text{cp}} \cdot \Delta q_2$. Without motor compensation, the spring extends:

$$\delta = r_{\text{cp}} \cdot \Delta q_2 \tag{10}$$

producing a parasitic restoring torque:

$$\tau_{\text{parasitic}} = k_{\text{exo}} \cdot r_{\text{cp}}^2 \cdot \Delta q_2 = k_{\text{eff}}^B \cdot \Delta q_2 \tag{11}$$

For $\Delta q_2 = 10$ (0.175 rad): $\tau_{\text{parasitic}} = 0.91 \times 0.175 = 0.16$ N m — clearly perceptible and unacceptable for "transparent" free rotation.

The tracking law is:

$$\theta_{m,R} = q_2 + \Delta\theta, \qquad \theta_{m,L} = -q_2 + \Delta\theta \tag{12}$$

where $\Delta\theta = 0$ for exo OFF (zero force) and $\Delta\theta > 0$ for co-contraction (symmetric pretension).

Contrast with Method A: In Method A, the springs sit at rest *passively* because the offset geometry does not couple q_2 rotation to spring extension (only the motor side of the cable connects to the offset pulley, and the spring is only loaded when the motor actively pulls). The encoder is an *upgrade*, not a requirement.

#	Discussion Point	Assessment
1	Two separate pulleys vs. one with two grooves	Nuanced
2	Stepper latency for zero-resistance feel	Nuanced
3	CubeMars impedance eliminating springs	Nuanced
4	"$2k$" co-contraction stiffness	Correct
5	Springs on the upper arm side	Incorrect
6	Transmission ratio via pulley diameters	Correct
7	Encoder tracking required for Method B	Correct

Table 1: Summary of meeting point assessments.

9 Summary of Assessments

10 Action Items

1. **Prototype decision:** Choose Method A or B for the first physical build. Method B is mechanically simpler (one pulley, exact kinematics) but requires reliable encoder tracking.
2. **Stepper vs. servo motor test:** Build a bench test with a stepper (or closed-loop stepper) tracking an encoder at $\dot{q}_2 \leq 5$ rad/s. Measure tracking latency and parasitic spring force.
3. **Spring placement:** Confirm springs are between elbow pulley exit and link-2 anchor (forearm side), not on the motor/upper-arm side.
4. **CubeMars integration path:** If CubeMars motors are used for the exo cables, implement hybrid control: physical springs for high-bandwidth impact response + motor torque mode for active stiffness modulation.
5. **Pulley diameter trade study:** Evaluate whether a non-unity transmission ratio ($r_m \neq r_{\text{cp}}$) improves motor torque headroom or resolution.
6. **Simulation validation:** Add exo cable dynamics to the Drake or Isaac Sim pipeline. Use the SEA infrastructure (`actuators/sea.py`) as a template for the exo cable spring-damper model.

11 Progress Update — 21 April 2026

Recorded meeting: `Audio-Weekly progress-20260421_115439-Meeting Recording.mp4`. Attendees: Debarup (in-person), Chris (remote, in lab), Ben (joined mid-meeting). Tube/rail strike restricted in-person attendance.

11.1 Arduino Motor Driver

- The wrong driver issue has been **resolved**.
- Two candidate Arduino libraries found for communicating with the DeBio motor controller via Arduino Uno:
  1. **Library 1** (PlatformIO-compatible, no English docs) — easy to install via PlatformIO but documentation is in Chinese only.
  2. **Library 2** (English docs, not on PlatformIO) — documentation available but requires manual installation.
- Most library function names are in English; inferred usage despite Chinese documentation. Example mistranslation: "point" rendered as "indicator".
- **Recommended translation tool: DeepL** (`https://www.deepl.com`) for technical documentation.

- **Recommended workflow:** Clone the library's GitHub repo, open in VS Code, and use GitHub Copilot to auto-generate English documentation from the English source code.

11.2 Physical Hardware — Cable Stoppers and Inserts

- Currently **3D-printing replacement parts** for the cable infrastructure (inserts and loop stoppers).
- Need a **cable-loop stopper** (white stopper previously used for the wire loop). Attempting to replicate with 3D-printed plastic; if insufficient, will order metal stoppers.
- Additional bolts and inserts may also need ordering; list to be sent by EOD 21 April.
- **Power supply issue:** Desktop power supply has non-standard plugs; may swap for a screw-terminal unit to connect motor and test rig.

11.3 Stepper vs. Servo Motor Decision

- Original plan was to use a **stepper motor** to track the joint encoder and maintain near-zero cable tension.
- Key concern: steppers are **not back-drivable** and can **miss steps** at high speed or under load—this adds perceptible resistance when moving the exo freely.
- **Servo motor alternative:** Peiyu (lab colleague) provided a servo motor (brought from China for personal use). This will be the first hardware under test.
- If the servo hypothesis validates, cheap but decent servo motors will be ordered from Robot Shop (fast delivery). The CubeMars motors are reserved for the manipulator, not the exo cables.
- The stepper latency test plan remains valid but the servo is now the **primary candidate**.

11.4 CubeMars Motor Procurement

- **Compliance certificate approved.**
- Two CubeMars motors ordered for the **manipulator** (not exo).
- Procurement path:
  1. 3DXR registered as a **university approved supplier** (ICT request submitted 21 April).
  2. HALA asked to create a **VCC (Virtual Credit Card)** for the order. VCC approval typically takes 4–5 days.
  3. Once VCC is issued, order placed; stock is available and delivery is ≈1–2 days.
- Motors feature a **planetary gearbox** (ratio $\approx 1:6$, back-drivable). These are the same class of motor used in humanoid/compliant robots.
- Exo cable motors remain a separate problem requiring the servo/stepper latency test.

11.5 GitHub Copilot Student Access

- Free access via the **GitHub Student Developer Pack**. Chris already has this. Strongly recommended for translating Chinese library docs and generating docstrings.

11.6 Lab Access — Robotics Area

- Ben (Chris?) submitted a request for **general robotics lab access** on 21 April. Parker (lab manager) to register them.
- General induction already completed. Access list update pending Becky's confirmation.
- Currently using the Ideas Lab upstairs mainly for 3D printer access; robotics area useful for hardware testing.

11.7 Exosuit Architecture — Key Design Discussion (with Ben)

Ben joined mid-meeting and described the wearable exosuit hardware. A major architectural discussion followed on how the exosuit, manipulator, and contraction-theory controller should be integrated.

Ben's Wearable Exosuit Hardware

- **Current wearable prototype:** single fixed elbow pulley + two separate braided cables routed to small motors. The motors can independently tension the exo springs.
- The two motors ("purple" cables in the diagram) *do not drive the joint* — they only add stiffness. The joint moves freely; the cables create a symmetric restoring force (co-contraction stiffness $k_{\text{eff}} = 2k_{\text{exo}}r_{\text{eff}}^2$).
- The adapter pulley is **mounted on the joint shaft**, so it rotates with the joint. Without motor compensation, this wraps/unwraps cable and stretches the springs (parasitic stiffness, see section 8). **Active encoder tracking nullifies this.**

Exosuit as a Signal Source (Not a Force Actuator)

The central design insight agreed in this meeting:

> **Key principle:** The exosuit is *not* meant to generate the required stiffness directly. It acts as a **stiffness signal source** that tells the manipulator/robot how much stiffness to generate. The manipulator's own back-drivable motors implement that stiffness via co-contraction.

Consequences:

- The exosuit motors need only be small and lightweight — they give a $\delta\theta$ command (tug), not a full torque command.
- When the manipulator already has the correct stiffness, the exosuit commands $\delta\theta = 0$ (stays passive).
- When a disturbance arrives and stiffness is insufficient, the exosuit activates: applies $q_2 \pm \delta\theta$ to stiffen the spring in the opposing direction. This is exactly the $\Delta\theta$ co-contraction command already implemented in simulation.
- Analogy used in meeting: "like a horse rider — you keep giving signals until the horse picks up speed, then you stop."

Three-Layer Control Architecture

The full system consists of three independent but cooperating layers:

1. **Motion control** (manipulator joint trajectories) — computed-torque / LQR controller.
2. **Stiffness control** (co-contraction via manipulator exo springs + back-drivable motors) — driven by the contraction-theory algorithm (`contraction-theory/`). This layer decides the correct k_{eff} given the disturbance.
3. **Exosuit signal layer** (wearable, lightweight) — a third add-on that monitors stiffness error and sends $\delta\theta$ signals to Layer 2 to "teach" it the correct stiffness level.

Impairment analogy: Treat the manipulator's stiffness controller as an "impaired brain" that does not know the optimal co-contraction level. The exosuit (Layer 3) is the rehabilitation tool: it provides an external stiffness signal until Layer 2 learns the correct response. Once Layer 2 converges, Layer 3 becomes passive.

Are Manipulator Springs Still Needed?

- If the manipulator uses **back-drivable motors** (e.g., CubeMars), physical springs in the cable path may be redundant — the motor itself provides programmable stiffness.
- Springs are still needed for the **exo cable motors** if those use steppers (not back-drivable).
- Conclusion: **springs may be removed from the manipulator cable path** once CubeMars motors are installed, but retained on the exo cable motors until a back-drivable alternative is confirmed.

Stiffness Modulation Ideas Discussed

Two lightweight alternatives to continuous motor pretension were discussed, motivated by the desire to make the wearable exosuit as small and power-efficient as possible.

Option A: PWM stiffness modulation. The motor drives the cable at full amplitude but is switched on/off at a high duty cycle $D \in [0, 1]$. When ON the spring is engaged at its full stiffness k_{exo}; when OFF the motor back-drives freely and the spring goes slack. Averaged over one PWM period, the effective joint stiffness is approximately:

$$k_{\text{eff,PWM}} \approx D \cdot 2\, k_{\text{exo}}\, r_{\text{cp}}^2 \tag{13}$$

so varying D from 0 to 1 continuously sweeps k_{eff} from zero to its maximum value.

Condition for validity: The PWM frequency f_{PWM} must be much higher than the mechanical bandwidth of the joint:

$$f_{\text{PWM}} \gg \frac{1}{2\pi}\sqrt{\frac{k_{\text{eff,max}}}{J_{\text{link}}}} \approx \frac{1}{2\pi}\sqrt{\frac{0.91}{0.05}} \approx 21 \text{ Hz} \tag{14}$$

so $f_{\text{PWM}} > 300$ Hz (well within electrical limits). However, the *mechanical* system (motor inertia + cable compliance) may not follow at that frequency, causing the joint to feel discrete on/off pulses as jitter rather than smooth intermediate stiffness. **Needs experimental validation.**

Option B: Clutch / brake-pad (binary stiffness switch). Replace the motor+spring with a **lockable cable mechanism** — a spring pre-tensioned at assembly, held free by a clutch. On command (solenoid, shape-memory wire, or micro-servo), the clutch engages and locks the cable, instantly loading the spring.

The torque applied to the joint becomes:

$$\tau_{\text{exo}} = \begin{cases} k_{\text{exo}}\, r_{\text{cp}}\left(r_{\text{cp}}\,\delta q - \delta_0\right) & \text{clutch ON (spring engaged)} \\ 0 & \text{clutch OFF (cable free)} \end{cases} \tag{15}$$

where δ_0 is the spring pre-compression at rest and δq is the joint deflection from nominal.

Effective behaviour:

- Engagement is *mechanical* speed — not limited by a control loop. Response is sub-millisecond for solenoid actuation.
- No continuous power is required to hold stiffness (spring is passive once locked).
- The energy absorbed when the clutch engages mid-motion goes into the spring (elastic storage), not heat — safe for wearable use.
- The on/off nature provides *bang-bang stiffness*: either full k_{eff} or zero. For the signal-source use case this is sufficient — the exosuit only needs to indicate "stiffen now", not modulate a precise level.

Comparison:
For a **lightweight wearable signal source**, the clutch option is the most attractive: a small solenoid (mass < 30 g) or shape-memory actuator can lock/unlock the cable, requiring only a brief current pulse (not sustained power) to change state.

Mechanism	Stiffness	Complexity	Weight	Power (hold)	Bandwidth
Motor + spring (current)	Continuous	High	Medium	High	Servo BW
PWM motor	Pseudo-variable	Medium	Medium	Medium	Needs test
Clutch / brake-pad	Binary (0 or k_s)	Low	Low	None	Mechanical

Table 2: Comparison of exosuit stiffness modulation options.

11.8 Agreed Next Steps

1. **[Chris, $\leq$28 Apr]** Complete stepper/servo latency test: measure tracking lag between joint encoder and exo motor at $\dot{q}_2 \leq 2$ rad/s. Use Peiyu's servo motor as first candidate. If lag is acceptable, order cheap servo motors from Robot Shop.

2. **[Chris, EOD 21 Apr]** Confirm 3D-printed cable-loop stopper feasibility; send purchase list for stoppers and bolts if 3D printing inadequate.

3. **[Chris]** Resolve Arduino library choice; use DeepL / Copilot for Chinese documentation.

4. **[Chris]** Register for GitHub Student Developer Pack.

5. **[Debarup]** Finalise VCC with HALA; order two CubeMars motors from 3DXR once ICT approves supplier.

6. **[Debarup/Ben]** Implement contraction-theory stiffness controller on the manipulator *first* (without exosuit), validate that it generates correct k_{eff} under disturbances.

7. **[All]** Add the wearable exosuit signal layer as a third add-on once Layers 1 and 2 are stable.

8. **[Ben]** Review CubeMars motor spec (planetary gear, 1:6 ratio, back-drivable) against exo motor requirements via links already shared.